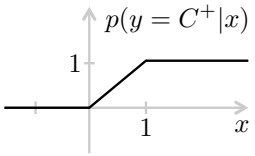
Q3. [9 pts] Decision Theory

We are given a test that is designed to predict whether a patient y has cancer C^+ or not C^-. The test returns a value $x \in \mathcal{R}$ and we know the probability of the patient having cancer given the test results:

$$p(y = C^+|x) = \begin{cases} 0, & \text{if } x < 0 \\ x, & \text{if } 0 \leq x < 1 \\ 1, & \text{if } 1 \leq x \end{cases}$$



We also know that it is three times more costly to have a false negative than a false positive. Specifically, the loss matrix is:

Predicted:

Truth:		C^-	C^+
	C^-	0	10
	C^+	30	0

Suppose that we choose a fixed value x^*, and we predict C^+ if the test result is greater than x^* and C^- otherwise.

(a) [2 pts] What is the decision boundary (value of x) that minimizes the misclassification probability?

$x = 1/2$ minimizes the classification boundary.

We want to choose x such that the probability of C^+ given x is the same as the probability of C^- given x:

$$P(y = C^+|x) = P(y = C^-|x)$$
$$x = 1 - x$$
$$x = 1/2$$

(b) [3 pts] What is the decision boundary (value of x^*) that minimizes the risk?

$x^* = 1/4$ minimizes the risk.

Choose x^* such that the risk of choosing C^+ given x is the same as the risk of choosing of C^- given x:

$$R(\hat{y} = C^+|x^*) = R(\hat{y} = C^-|x^*)$$
$$\ell(C^+, C^+)P(y = C^-|x^*) + \ell(C^+, C^-)P(y = C^-|x^*) = \ell(C^-, C^+)P(y = C^+|x^*) + \ell(C^-, C^-)P(y = C^-|x^*)$$
$$\ell(C^+, C^-)P(y = C^-|x^*) = \ell(C^-, C^+)P(y = C^+|x^*)$$
$$10(1 - x^*) = 30x^*$$
$$x^* = 1/4$$

(c) [4 pts] If the test result is uniformly distributed in the interval $[-1, 1]$, what is the value of the minimum risk? Write your answer in terms of x^* (to avoid loss of points if your x^* is incorrect).

$$
\begin{aligned}
\mathrm{E}\ell(\hat{y}, y) &= \mathrm{EE}[\ell(\hat{y}, y)|x] \\
&= \int_{-\infty}^{\infty} \mathbb{1}[\hat{y} = C^-]\ell(C^-, C^+)P(y = C^+, x) + \mathbb{1}[\hat{y} = C^+]\ell(C^+, C^-)P(y = C^-, x)dx \\
&= \int_{-\infty}^{\infty} \mathbb{1}[\hat{y} = C^-]\ell(C^-, C^+)P(y = C^+|x)P(x) + \mathbb{1}[\hat{y} = C^+]\ell(C^+, C^-)P(y = C^-|x)P(x)dx \\
&= \int_{0}^{x^*} \frac{1}{2}\ell(C^-, C^+)P(y = C^+|x)dx + \int_{x^*}^{1} \frac{1}{2}\ell(C^+, C^-)P(y = C^-|x)dx \\
&= \frac{1}{2}\int_{0}^{x^*} 30x dx + \int_{x^*}^{1} 10(1 - x)dx \\
&= \frac{1}{2}\left(15x^2\Big]_0^{x^*} + 10x - 5x^2\Big]_{x^*}^{1}\right) \\
&= \frac{1}{2}\left(15x^{*2} + 5 - 10x^* + 5x^{*2}\right) \\
&= 10x^{*2} - 5x^* + 2.5 = \frac{15}{8}
\end{aligned}
$$

Q4. [8 pts] Parameter Estimation

Suppose you are given n observations, $X_1, ..., X_n$, independent and identically distributed with a Gamma(α, λ) distribution. The following information might be useful for one or more parts of the problem.

- If $X \sim \text{Gamma}(\alpha, \lambda)$, then $\mathbb{E}[X] = \frac{\alpha}{\lambda}$ and $\mathbb{E}[X^2] = \frac{\alpha(\alpha+1)}{\lambda^2}$
- The probability density function of $X \sim \text{Gamma}(\alpha, \lambda)$ is $f_X(x) = \frac{1}{\Gamma(\alpha)}\lambda^\alpha x^{\alpha-1}e^{-\lambda x}$ where the function Γ is only dependent on α and not λ.

The following notation might be useful for one or more parts of the problem: $\overline{X}_1 = \frac{1}{n}\sum_{i=1}^{n} X_i$ and $\overline{X}_2 = \frac{1}{n}\sum_{i=1}^{n} X_i^2$.

(a) [4 pts] Find the estimators for α and λ using the method of moments. Remember you are trying to write α and λ as functions of the data.

First, we take the sample moments from the data, $\overline{X}_1, \overline{X}_2$ and equate that to the theoretical moments, $\mathbb{E}[X], \mathbb{E}[X^2]$.

$$\overline{X}_1 = \frac{\alpha}{\lambda}$$
$$\overline{X}_2 = \frac{\alpha(\alpha+1)}{\lambda^2}$$

Solving for α and λ from this system of equations, we get

$$\hat{\alpha}_{\text{MM}} = \frac{\overline{X}_1^2}{\overline{X}_2 - \overline{X}_1^2} = \frac{\overline{X}_1^2}{\hat{\sigma}^2}$$
$$\hat{\lambda}_{\text{MM}} = \frac{\overline{X}_1}{\overline{X}_2 - \overline{X}_1^2} = \frac{\overline{X}_1}{\hat{\sigma}^2}$$

(b) [4 pts] Suppose, we are given a known, fixed value for α. Compute the maximum likelihood estimator for λ.

We first write the likelihood function.

$$L(\lambda | X_1, ..., X_n) = \prod_{i=1}^{n} \frac{1}{\Gamma(\alpha)}\lambda^\alpha X_i^{\alpha-1}e^{-\lambda X_i}$$

The log-likelihood function is given as follows.

$$l(\lambda | X_1, ..., X_n) = -n\log(\Gamma(\alpha)) + n\alpha\log\lambda + \sum_{i=1}^{n}(\alpha-1)\log X_i - \lambda X_i$$

Next, we take the gradient with respect to λ and set it equal to 0.

$$\nabla_\lambda l(\lambda | X_1, ..., X_n) = \frac{n\alpha}{\lambda} - \sum_{i=1}^{n} X_i = 0$$

Solving for λ, we get,

$$\hat{\lambda}_{\text{MLE}} = \frac{n\alpha}{\sum_{i=1}^{n} X_i} = \frac{\alpha}{\frac{1}{n}\sum_{i=1}^{n} X_i} = \frac{\alpha}{\overline{X}_1}$$

Q5. [14 pts] Locally Weighted Logistic Regression

In this problem, we consider solving the problem of locally weighted logistic regression. Given data $\{(x_i, y_i) \in \mathbb{R}^d \times \{0, 1\}\}_{i=1}^n$ and a query point x, we choose a parameter vector θ to minimize the loss (which is simply the negative log likelihood, weighted appropriately):

$$l(\theta; x) = -\sum_{i=1}^{n} w_i(x) \left[y_i \log(\mu(x_i)) + (1 - y_i) \log(1 - \mu(x_i)) \right]$$

where

$$\mu(x_i) = \frac{1}{1 + e^{-\theta \cdot x_i}}, \qquad w_i(x) = \exp\left(-\frac{\|x - x_i\|^2}{2\tau} \right)$$

where τ is a hyperparameter that must be tuned. Note that whenever we receive a new query point x, we must solve the entire problem again with these new weights $w_i(x)$.

Hint: the derivative of the logistic regression log likelihood with respect to θ is: $\sum_{i=1}^{n}(y_i - \mu(x_i))x_i$

(a) [4 pts] Given a data point x, derive the gradient of $l(\theta; x)$ with respect to θ.

The derivation is extremely similar to the derivation of logistic regression as in the slides. The answer is

$$-X^\intercal z = -\sum_{i=1}^{n} w_i(x)(y_i - \mu(x_i))x_i$$

where X is the design matrix (i.e. every row in X is a data point), and $z_i = w_i(x)(y_i - \mu(x_i))$.

(b) [4 pts] Given a data point x, derive the Hessian of $l(\theta; x)$ with respect to θ.

The derivation is extremely similar to the derivation of logistic regression as in the slides. The answer is

$$X^T D X = \sum_{i=1}^{n} w_i(x)\mu(x_i)(1 - \mu(x_i))x_i x_i^\intercal$$

(c) [2 pts] Given a data point x, write the update formula for gradient descent. Use the symbol η for an arbitrary step size.

$$\theta^{(t+1)} = \theta^{(t)} + \eta X^\intercal z$$

(d) [2 pts] Given a data point x, write the update formula for Newton's method.

$$\theta^{(t+1)} = \theta^{(t)} + [X^T D X]^{-1} X^\intercal z$$

(e) [2 pts] Locally Weighted Logistic Regression is a

○ Parametric method ● Nonparametric method

Q6. [7 pts] Decision Trees

Answer the following questions related to decision trees.

(a) [3 pts] In Homework 5, you first implemented a decision tree and then implemented a decision forest, which uses an ensemble method called bagging.

For neural network classification, it is typical to train k networks and average the results. Why not run your decision tree (using all of the data) k times and then average the results?

Decision trees return the same result each time you run them.

(b) [2 pts] True or false: Selecting the decision tree split (at each node as you move down the tree) that *minimizes classification error* will guarantee an optimal decision tree.

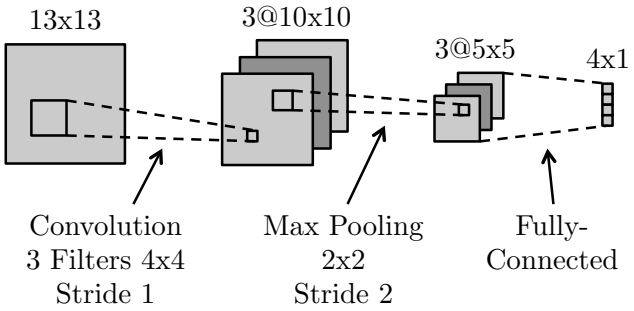
◯ True ● False

(c) [2 pts] True or false: Selecting the decision tree split (at each node as you move down the tree) that *maximizes information gain* will guarantee an optimal decision tree.

◯ True ● False

Q7. [11 pts] Convolutional Neural Nets

Below is a diagram of a small convolutional neural network that converts a 13x13 image into 4 output values. The network has the following layers/operations from input to output: convolution with 3 filters, max pooling, ReLu, and finally a fully-connected layer. For this network we will not be using any bias/offset parameters (b). Please answer the following questions about this network.



(a) [2 pts] How many weights in the convolutional layer do we need to learn?

48 weights. Three filters with 4x4=16 weights each.

(b) [2 pts] How many ReLu operations are performed on the forward pass?

75 ReLu operations. ReLu is performed after the pooling step. ReLu is performed on each pixel of the three 5x5 feature images.

(c) [2 pts] How many weights do we need to learn for the entire network?

348 weights. 48 for the convolutional layer. Fully-connected has 3x5x5=75 pixels each connected to four outputs, which is 300 weights. Pooling layer does not have any weights.

(d) [2 pts] True or false: A fully-connected neural network with the same size layers as the above network (13x13 $\rightarrow$ 3x10x10 $\rightarrow$ 3x5x5 $\rightarrow$ 4x1) can represent any classifier that the above convolutional network can represent.

● True ○ False

(e) [3 pts] What is the disadvantage of a fully-connected neural network compared to a convolutional neural network with the same size layers?

Too many weights to effectively learn.

Q8. [9 pts] Streaming k-means

The standard k-means algorithm loads all data points altogether into the memory. In practice, data usually comes in a stream, such that they are sequentially processed and dropped (not stored in memory). The advantage of streaming algorithms is that their memory requirement is independent of the stream length. Thus, streaming algorithms are very useful in processing data that cannot fit into the memory.

In this problem, we will explore how to extend the k-means algorithm to process streaming data. Suppose that there are k clusters. The cluster centers are randomly initialized. Once the processor receives a data point $x \in \mathbb{R}^d$, it does the following:

1. Find the cluster whose center is the closest to x (in Euclidean distance), then add x to the cluster
2. Adjust the cluster center so that it equals the mean of all cluster members.

The algorithm outputs the k cluster centres after processing all data points in the stream.

According to the above algorithm specification, complete the streaming algorithm for k-means. Note that the algorithm's memory requirement should be independent of the stream length.

(a) [3 pts] List the variables that are stored in the memory and their initial values. Which variables should be the output of the algorithm?

Two sets of variables need to be stored:

- c_i: the center of the i-th cluster ($i = 1, \ldots, k$). It is a d-dimensional vector and should be randomly initialized in $\mathbb{R}^d$.
- n_i: the number of data points that belong to the i-th cluster. It should be initialized by 0.

The algorithm's output should be c_i ($i = 1, \ldots, k$).

(b) [3 pts] When the processor receives a data point x, state the updates that are made on the variables.

The algorithm executes the following three steps:

1. Find the index i which minimizes $\|x - c_i\|_2$ for $i = 1, \ldots, k$.
2. Update $c_i \leftarrow (n_i c_i + x)/(n_i + 1)$.
3. Update $n_i \leftarrow n_i + 1$.

(c) [3 pts] In each iteration, suppose the processor receives a data point x along with its weight $w > 0$. We want the cluster center to be the weighted average of all cluster members. How do you modify the updates in question (b) to process weighted data?

With weighted data, the algorithm executes the following three steps:

1. Find the index i which minimizes $\|x - c_i\|_2$ for $i = 1, \ldots, k$.
2. Update $c_i \leftarrow (n_i c_i + wx)/(n_i + w)$.
3. Update $n_i \leftarrow n_i + w$.